

LLM Tutorial

AI in Healthcare

MSAI Spr '25

Chelsea Ramos

Overview

- Healthcare Issue
- Methods Employed
- Setting Up
- Medical Dataset Used
 - Load Data
 - Clean & Merge Data
- Prompts Engineered
- Binary Classification
- Evaluation of Results
- Ideas for Improvement

Healthcare Issue

Predict whether a patient needs a prediabetes care plan based on demographics and clinical observations.

- A prediabetes care plan can significantly reduce the risk of developing type 2 diabetes.
- Accurate decisions can help save lives and reduce healthcare costs.

Methods Employed

1. I'll evaluate In-Context Learning methods:
 - Zero-Shot Learning (instructions only, no examples)
 - Chain-of-Thought (include step-by-step reasoning)
 - Few-Shot Learning (include example)
 - Tree of Thought (evaluate multiple reasoning paths)
2. Then try Binary Classification w/ embeddings generated via LLM

Setting Up

Model: SmolLM2 from Hugging Face

```
# Imports
import pandas as pd
import torch
from IPython.display import display_html
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.model_selection import train_test_split
from tqdm import tqdm
from transformers import AutoModelForCausalLM, AutoTokenizer
```

✓ 3.5s

```
# Load model from HuggingFace
checkpoint = "HuggingFaceTB/SmolLM2-360M-instruct" # Smaller model for demo
# checkpoint = "HuggingFaceTB/SmolLM2-1.7B-instruct" # Larger model for if you have enough resources

device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint).to(device)
```

✓ 3.0s

Medical Dataset Used

- Publicly available synthesized data (7 MB of data with 100 patients)
 - Tables used (3):
 - patients
 - observations
 - careplans
- Download ***Latest Version of Synthea: 100 Sample Synthetic Patient Records, CSV: 7 MB*** from <https://synthea.mitre.org/downloads>
- Save and unzip your downloaded ***synthea_sample_data_csv_latest.zip*** to a data path you can access via this notebook.

Load Data

```
# Set your data path:
DATA_PATH = '../data/llm/synthea_sample_data_csv_latest/'

# Helper function for data loading and previewing
def load_and_preview_data(dfs, csv_filename):
    df = pd.read_csv(DATA_PATH + csv_filename + '.csv')
    print(csv_filename.capitalize(), 'data loaded. Shape:', df.shape)
    display_html(df.head())
    dfs[csv_filename] = df
    return dfs

# Specify desired tables
dfs_to_load = [
    'patients',
    'observations',
    'careplans',
]

dfs = {} # Store dataframes here
# Load the dataframes
for df in dfs_to_load:
    dfs = load_and_preview_data(dfs, df)
```

Patients data loaded. Shape: (106, 28)

	Id	BIRTHDATE	DEATHDATE
0	30a6452c-4297-a1ac-977a-6a23237c7b46	1994-02-06	NaN
1	34a4dcc4-35fb-6ad5-ab98-be285c586a4f	1968-08-06	2009-12-11
2	7179458e-d6e3-c723-2530-d4acfe1c2668	2008-12-21	NaN
3	37c177ea-4398-fb7a-29fa-70eb3d673876	1994-01-27	NaN
4	0fef2411-21f0-a269-82fb-c42b55471405	2019-07-27	NaN

5 rows x 28 columns

Observations data loaded. Shape: (86360, 9)

	DATE	
0	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-
1	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-
2	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-
3	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-
4	2016-04-10T09:04:48Z	30a6452c-4297-a1ac-

Careplans data loaded. Shape: (329, 9)

	Id	START	S
0	47441dae-e581-6d99-e5fc-b7fbd9cde7fe	2015-09-28	21

Clean & Merge Data Pt. 1

Patients

```
df = dfs['patients']
# Reduce cols
cols = [
    'Id',
    'BIRTHDATE',
    'MARITAL',
    'RACE',
    'ETHNICITY',
    'GENDER',
    'HEALTHCARE_EXPENSES',
    'HEALTHCARE_COVERAGE',
    'INCOME'
]
df = df[cols]
dfs['patients'] = df
# Add new feature cols
dfs['patients']['expenses_covered'] = dfs['patients']['HEALTHCARE_COVERAGE'] > dfs['patients']['HEALTHCARE_EXPENSES']
dfs['patients']['high_income'] = dfs['patients']['INCOME'] > 50000
dfs['patients'] = dfs['patients'].drop(columns=['HEALTHCARE_EXPENSES', 'HEALTHCARE_COVERAGE', 'INCOME'])
print(dfs['patients'].shape)
dfs['patients'].head()
```

✓ 0.0s

(106, 8)

	Id	BIRTHDATE	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income
0	30a6452c-4297-a1ac-977a-6a23237c7b46	1994-02-06	M	white	nonhispanic	M	False	True

Observations

```
df = dfs['observations']
# Reduce cols
df = df[['PATIENT', 'DATE', 'CATEGORY', 'DESCRIPTION', 'CODE', 'VALUE', 'UNITS']]
dfs['observations'] = df
# Rename cols
dfs['observations'] = dfs['observations'].rename(
    columns={
        'DATE': 'ObservationDate',
        'CATEGORY': 'ObservationCategory',
        'DESCRIPTION': 'ObservationDescription',
        'VALUE': 'ObservationValue',
        'UNITS': 'ObservationUnits'
    }
)
print(dfs['observations'].shape)
dfs['observations'].head()
```

✓ 0.0s

(86360, 7)

	PATIENT	ObservationDate	ObservationCategory
0	30a6452c-4297-a1ac-977a-6a23237c7b46	2016-04-10T09:04:48Z	vital-signs

Clean & Merge Data Pt. 2

Careplans

```
df = dfs['careplans']
# Reduce cols
df = df[['PATIENT', 'START', 'REASONDESCRIPTION']]
dfs['careplans'] = df
# Keep last careplan per patient
dfs['careplans'] = dfs['careplans'].sort_values(
    by=['PATIENT', 'START'], ascending=[True, True]
).drop_duplicates(subset=['PATIENT'], keep='last')
print(dfs['careplans'].shape)
# Fill NA values
dfs['careplans']['REASONDESCRIPTION'] = dfs['careplans']['REASONDESCRIPTION'].fillna('NA')
# Create binary column for prediabetes
dfs['careplans']['PREDIABETES'] = dfs['careplans']['REASONDESCRIPTION'].apply(
    lambda x: 1 if 'prediabetes' in x.lower() else 0
)
dfs['careplans'] = dfs['careplans'].drop(columns=['REASONDESCRIPTION'])
dfs['careplans'] = dfs['careplans'].rename(
    columns={'START': 'PREDIABETES_START'})
dfs['careplans'].head()
```

✓ 0.0s

(100, 3)

	PATIENT	PREDIABETES_START	PREDIABETES
37	00732e11-5e4d-37b7-01f8-929a25536862	2023-12-20	0

Merge Data

```
final_df = dfs['careplans'].copy()
# Merge dataframes
final_df = final_df.merge(
    dfs['patients'],
    left_on='PATIENT',
    right_on='Id',
    how='inner'
)
final_df = final_df.merge(
    dfs['observations'],
    left_on='PATIENT',
    right_on='PATIENT',
    how='inner'
)
# Add age column
final_df['age'] = pd.to_datetime(
    final_df['PREDIABETES_START']).dt.year - pd.to_datetime(final_df['BIRTHDATE']).dt.year
# Reduce cols
final_df = final_df.drop(columns=['BIRTHDATE', 'PREDIABETES_START', 'Id'])
# Fix ObservationValues
final_df['ObservationValue'] = pd.to_numeric(final_df['ObservationValue'], errors='coerce')
print(final_df.shape)
final_df.head()
```

✓ 0.0s

(85395, 15)

	PATIENT	PREDIABETES	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income
0	00732e11-5e4d-37b7-01f8-929a25536862	0	W	white	nonhispanic	M	True	False

Prompts Engineered

Patient #1 – Needs Prediabetes Care Plan

```
# Get patient who does require a prediabetes care plan
# Define conditions for prediabetes
bmi_cond = final_df[(final_df['CODE'] == '39156-5') &
                    (final_df['ObservationValue'] >= 30)]
a1c_cond = final_df[(final_df['CODE'] == '4548-4') &
                    (final_df['ObservationValue'].between(5.7, 6.4))]
# Get patients matching each condition
bmi_patients = set(bmi_cond['PATIENT'])
a1c_patients = set(a1c_cond['PATIENT'])
# Find patients who match multiple conditions
matching_patients = bmi_patients & a1c_patients
# Filter original dataframe
final_df_matching = final_df[final_df['PATIENT'].isin(matching_patients)]
# Get 1 patient
final_df_matching.loc[final_df_matching['PREDIABETES'] == 1]['PATIENT'].unique()[-1]
```

✓ 0.0s

Python

'65016a46-14f4-d19a-f82f-10299aba4c14'

```
patient = '65016a46-14f4-d19a-f82f-10299aba4c14'
final_df_matching.loc[(final_df_matching['PATIENT'] == patient) &
                    ((final_df['CODE'] == '39156-5') |
                     (final_df['CODE'] == '4548-4') |
                     (final_df['CODE'] == '2571-8'))].sort_values(
    by=['ObservationDate'], ascending=[True]
).drop_duplicates(subset=['ObservationDescription'], keep='last').drop(columns=['PATIENT', 'PREDIABETES', 'ObservationDate'])
```

✓ 0.0s

Python

	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	ObservationCategory	ObservationDescription	CODE	ObservationValue	ObservationUnits	age
22300	S	white	nonhispanic	F	False	True	laboratory	Triglycerides	2571-8	125.4	mg/dL	34
22372	S	white	nonhispanic	F	False	True	laboratory	Hemoglobin A1c/Hemoglobin.total in Blood	4548-4	6.0	%	34
22376	S	white	nonhispanic	F	False	True	vital-signs	Body mass index (BMI) [Ratio]	39156-5	30.1	kg/m2	34

Zero-Shot Learning

– I developed a simple question based on Patient #1's data:

```
patient1_info = """
    They are single, white, nonhispanic, female, 34 years old, high income, and have expenses not covered,
    Triglycerides 125.4 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.0%, and Body mass index (BMI) [Ratio] 30.1 kg/m2.
    """

prompt = f"""
    Decide in a single word if a patient requires a prediabetes care plan or not.
    The patient has the following information:
    {patient1_info}
    """

messages = [
    {"role": "system",
     "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care plan or not."},
    {"role": "user",
     "content": prompt}
]

input_text=tokenizer.apply_chat_template(messages, tokenize=False)
inputs = tokenizer.encode(input_text, return_tensors="pt").to(device)
outputs = model.generate(
    inputs,
    max_new_tokens=50,
    temperature=0.2,
    top_p=0.9,
    do_sample=True)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

– Response:

No, the patient does not require a prediabetes care plan.

– This was incorrect.

Chain-of-Thought: Prompting

- This time, I asked to "think first" and include reasoning for CoT:

```
prompt1 = f"""
Think first and explain your reasoning on whether a patient requires a
prediabetes care plan or not. Then, give a final decision.
The patient has the following information:
{patient1_info}
"""
```

```
messages = [
    {"role": "system",
     "content": "You are a doctor and an expert on deciding if a patient
requires a prediabetes care plan or not."},
    {"role": "user",
     "content": prompt1},
]
```

Chain-of-Thought: Response

Based on the patient's information, I would recommend that they require a prediabetes care plan. The patient's high income, high expenses **not** covered, **and** high BMI indicate that they may be at risk for developing prediabetes. Prediabetes is a condition where the body's blood sugar levels are higher than normal, but **not** high enough to be classified as diabetes. This condition increases the risk of developing type 2 diabetes, heart disease, **and** other health problems.

The patient's high cholesterol levels, high triglycerides, **and** high blood pressure also suggest that they may be at risk for developing these conditions. A prediabetes care plan would focus on managing these conditions through lifestyle changes, such as diet **and** exercise, **and** potentially medication **if** necessary.

Additionally, the patient's age (34 years old) **and** gender (female) may also indicate that they are at higher risk for developing prediabetes. Prediabetes is more common in women **and** older adults, **and** it's estimated that up to 1 in 4 women may develop prediabetes by the age of 65.

Based on these factors, a prediabetes care plan would be recommended for this patient. This plan would aim to help them manage their cholesterol, triglycerides, **and** blood pressure, **and** to maintain a healthy weight through a combination of diet **and** exercise. It would also include regular **check-ups** to **monitor** their progress **and** adjust their plan as needed.

- CoT helped to get to the correct conclusion!
- However, a lot of statements were made that may not be true (hallucinations)

Patient #2 – Prediabetes Care Plan Not Needed

```
# Get patient who does not require a prediabetes care plan
final_df.loc[(final_df['PREDIABETES'] == 0) & (final_df['age'].between(25, 35)) &
             (final_df['CODE'].isin(['4548-4']))].sample(1, random_state=777)['PATIENT'].unique()
```

✓ 0.0sPython

array(['4804956b-3c8f-baa7-9a42-87518e486055'], dtype=object)

```
patient = '4804956b-3c8f-baa7-9a42-87518e486055'
final_df.loc[(final_df['PATIENT'] == patient) &
             ((final_df['CODE'] == '39156-5') |
              (final_df['CODE'] == '4548-4') |
              (final_df['CODE'] == '2571-8'))].sort_values(
    by=['ObservationDate'], ascending=[True]
).drop_duplicates(subset=['ObservationDescription'], keep='last').drop(columns=['PATIENT', 'PREDIABETES', 'ObservationDate'])
```

✓ 0.0sPython

	MARITAL	RACE	ETHNICITY	GENDER	expenses_covered	high_income	ObservationCategory	ObservationDescription	CODE	ObservationValue	ObservationUnits	age
12998	M	white	nonhispanic	F	True	False	laboratory	Hemoglobin A1c/Hemoglobin.total in Blood	4548-4	6.2	%	33
13002	M	white	nonhispanic	F	True	False	vital-signs	Body mass index (BMI) [Ratio]	39156-5	28.3	kg/m2	33
13016	M	white	nonhispanic	F	True	False	laboratory	Triglycerides	2571-8	128.0	mg/dL	33

– Next, I developed a prompt for Patient #2:

```
patient2_info = """
    They are married, white, nonhispanic, female, 33 years old, not high income, and have expenses covered,
    Triglycerides 128.0 mg/dL, Hemoglobin A1c/Hemoglobin.total in Blood 6.2%, and Body mass index (BMI) [Ratio] 28.3
    kg/m2.
    """

prompt2 = f"""
    Think first and explain your reasoning on whether a patient requires a prediabetes care plan or not. Then, give a
    final decision.
    The patient has the following information:
    {patient2_info}
    """
```

Chain-of-Thought + Few-Shot Learning: Prompting

- Continuing the CoT conversation w/ Patient #1 (`prompt1`), I provided an example `response1` before `prompt2`:

```
response1 = """
    1. Triglycerides 125.4 mg/dL is within the normal range (less than 150 mg/dL).
    2. Hemoglobin A1c/Hemoglobin.total in Blood 6.0% is between 5.7% to 6.4%, which is
considered prediabetes.
    3. Body mass index (BMI) [Ratio] 30.1 kg/m2 is considered obese (BMI >= 30).
    Based on the patient's information, I would recommend that they require a prediabetes
care plan.
    """
messages = [
    {"role": "system",
     "content": "You are a doctor and an expert on deciding if a patient requires a
prediabetes care plan or not."},
    {"role": "user",
     "content": prompt1},
    {"role": "assistant", "content": response1},
    {"role": "user",
     "content": prompt2},
]
```


Chain-of-Thought + Few-Shot Learning: Response

1. Triglycerides 128.0 mg/dL is within the normal range (less than 150 mg/dL).

2. Hemoglobin A1c/Hemoglobin.total in Blood 6.2% is between 5.4% to 6.6%, which is considered prediabetes.

3. Body mass index (BMI) [Ratio] 28.3 kg/m2 is considered normal (BMI ≥ 25).

Based on the patient's information, I would recommend that they require a prediabetes care plan.

- This conclusion was incorrect, but Few-Shot Learning helped to improve the structure of the response greatly!
- Good structure helps eliminate hallucinations

Tree of Thought: Prompting

– Last, I asked to consider multiple reasoning paths for ToT:

```
# Prompt engineered w/ the help of ChatGPT 4o:
prompt3 = f"""
    Task: Decide if a patient needs a prediabetes care plan.

    Step 1: Generate 3 possible reasoning paths.
    Option A: ...
    Option B: ...
    Option C: ...

    Step 2: Evaluate each option for correctness and relevance.
    Evaluation: ...

    Step 3: Select or combine the best option.
    Final Answer: ...

    The patient has the following information:
    {patient2_info}
"""
messages = [
    # {"role": "system",
    #   "content": "You are a doctor and an expert on deciding if a patient requires a prediabetes care
    plan or not."},
    {"role": "user",
     "content": prompt3},
]
```

Tree of Thought: Response

Option A: The patient does **not** need **a** prediabetes care plan.

Reasoning: The patient has **a** healthy lifestyle, no high-risk factors, **and** is **not** high-income. These factors **do not** indicate **the** need **for a** prediabetes care plan.

Option B: The patient needs **a** prediabetes care plan.

Reasoning: The patient has high triglycerides, high blood sugar, **and** high BMI, which are all risk factors **for** prediabetes. These factors indicate **the** need **for a** prediabetes care plan.

Option C: The patient does **not** need **a** prediabetes care plan.

Reasoning: The patient has **a** healthy lifestyle, no high-risk factors, **and** is **not** high-income. These factors **do not** indicate **the** need **for a** prediabetes care plan.

Final Answer: The patient does **not** need **a** prediabetes care plan.

- The correct conclusion was reached!
- ToT helped more than CoT for this patient

Binary Classification

**Using embeddings
generated via LLM**

Generating Texts from Data Pt. 1

- For modeling, we need text embeddings
- First, data must become texts:

MARITAL: W, RACE: white, ETHNICITY: nonhispanic, GENDER: M, expenses_covered: True, high_income: False, age: 48, Hemoglobin A1c/Hemoglobin.total in Blood: 2.9 %, Body mass index (BMI) [Ratio]: 29.4 kg/m2, Triglycerides: 132.0 mg/dL

```
# Prepare data for LLM & classification
class_df = final_df.copy()
class_df = class_df.loc[(class_df['CODE'] == '39156-5') |
                        (class_df['CODE'] == '4548-4') |
                        (class_df['CODE'] == '2571-8')].copy()
# Reduce cols
class_df['Observation'] = (class_df['ObservationValue'].astype(str)
                           + ' ' + class_df['ObservationUnits'].astype(str))
# Aggregate multiple col info into text
cols = list(class_df.columns[2:8]) + ['age']
def row_to_text_auto(row):
    return ", ".join([f"{col}: {row[col]}" for col in cols])
class_df['text'] = class_df.apply(row_to_text_auto, axis=1)
class_df = class_df[['PREDIABETES', 'text', 'PATIENT', 'CODE',
                    'ObservationDate', 'ObservationDescription',
                    'ObservationValue', 'ObservationUnits']]
print(class_df['text'][0])
class_df.head(1)
```

✓ 0.0s

Python

MARITAL: W, RACE: white, ETHNICITY: nonhispanic, GENDER: M, expenses_covered: True, high_income: False, age: 48

PREDIABETES		text	PATIENT	CODE	ObservationDate	ObservationDescription	ObservationValue	ObservationUnits
0	0	MARITAL: W, RACE: white, ETHNICITY: nonhispani...	00732e11- 5e4d-37b7- 01f8- 929a25536862	4548- 4	2015-06- 24T14:05:28Z	Hemoglobin A1c/Hemoglobin.total in Blood	3.3	%

Generating Texts from Data Pt. 2

```
# Add observations to text
class_df1 = class_df[['PREDIABETES', 'text', 'PATIENT']].copy()
class_df2 = class_df[['PATIENT', 'ObservationDate', 'CODE',
                      'ObservationDescription', 'ObservationValue',
                      'ObservationUnits']].copy()
for patient in class_df1['PATIENT'].unique():
    cur_df2 = class_df2.loc[(class_df2['PATIENT'] == patient) &
                           ((class_df2['CODE'] == '39156-5') |
                            (class_df2['CODE'] == '4548-4') |
                            (class_df2['CODE'] == '2571-8'))].sort_values(
        by=['ObservationDate'], ascending=[True]
    ).drop_duplicates(subset=['ObservationDescription'], keep='last')
    for obs in cur_df2.itertuples():
        # Add the observation the text
        class_df1.loc[class_df1['PATIENT'] ==
                      patient,
                      'text'] += f", {obs.ObservationDescription}: {obs.ObservationValue} {obs.ObservationUnits}"
print(class_df1['text'][0])
class_df1 = class_df1.drop(columns=['PATIENT'])
class_df1.head(1)
```

✓ 0.1s

Python

MARITAL: W, RACE: white, ETHNICITY: nonhispanic, GENDER: M, expenses_covered: True, high_income: False, age: 48, Hemoglobin

PREDIABETES

text

0

0

MARITAL: W, RACE: white, ETHNICITY: nonhispani...

Generating Embeddings from Texts Pt. 1

```
# Generate embeddings for classification
# Code adapted from lecture w/ help from ChatGPT 4o
def mean_pooling(hidden_states, attention_mask):
    mask = attention_mask.unsqueeze(-1).expand(hidden_states.size()).float()
    return (hidden_states * mask).sum(dim=1) / mask.sum(dim=1)
def get_embeddings(texts, batch_size=16):
    embeddings = []
    for i in tqdm(range(0, len(texts), batch_size)):
        batch = texts[i:i+batch_size]
        encodings = tokenizer(
            batch,
            return_tensors='pt',
            padding=True,
            truncation=True,
            max_length=512
        )
        encodings = {k: v.to(device) for k, v in encodings.items()}
        with torch.no_grad():
            outputs = model(**encodings, output_hidden_states=True)
            hidden_states = outputs.hidden_states[-1]
            pooled = mean_pooling(hidden_states, encodings['attention_mask'])
            embeddings.append(pooled.cpu())
    return torch.cat(embeddings, dim=0).numpy()
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    class_df1['text'], class_df1['PREDIABETES'], test_size=0.2, random_state=42
)
# Generate embeddings
X_train = get_embeddings(X_train.tolist())
X_test = get_embeddings(X_test.tolist())
# Extract labels
y_train = y_train.values
y_test = y_test.values
```

✓ 2m 58.1s

Generating Embeddings from Texts Pt. 2

- Now, embeddings can be generated from texts
- This can be time consuming, but luckily there are only ~100 patients

```
100%|██████████| 128/128 [02:26<00:00, 1.15s/it]  
100%|██████████| 32/32 [00:30<00:00, 1.06it/s]
```


Classification Results

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
✓ 0.1s
```

▼ LogisticRegression ⓘ ⓘ
LogisticRegression(max_iter=1000)

```
test_pred = model.predict_proba(X_test)[: ,1]
auroc = roc_auc_score(y_test, test_pred)
auprc = average_precision_score(y_test, test_pred)
print('\nAUROC:', auroc, '\nAUPRC', auprc)
✓ 0.0s
```

AUROC: 0.9999999999999999
AUPRC 1.0

- Text embeddings worked well for classification, even with a simple model

Evaluation of Results

- In-Context Learning
 - Zero-Shot Learning (instructions only, no examples)
 - Did not produce a correct conclusion, but was a great starting point
 - Chain-of-Thought (include step-by-step reasoning)
 - Helped to reach correct conclusion for Patient #1, but produced some meandering thoughts/hallucinations
 - Few-Shot Learning (include example)
 - (Combined w/ CoT) Produced incorrect conclusion, but improved structure greatly and therefore eliminated hallucinations
 - Tree of Thought (evaluate multiple reasoning paths)
 - Helped (more than CoT) to reach correct conclusion for Patient #2
- Binary Classification w/ embeddings generated via LLM
 - Achieved near-perfect results w/ simple Logistic Regression
 - Using text embeddings reduces data cleaning and feature engineering steps greatly, but more compute time/resources are needed to generate embeddings

Ideas for Improvement

- Try larger or medically fine-tuned LLMs
 - These could yield more accurate conclusions with less reliance on prompt engineering
- Experiment w/ multiple LLMs (2–3)
 - Known as "LLM ensembling," inspired by ML ensembles, which generally produce better results than individual models
- Try OpenAI APIs or similar cloud-based models
 - To overcome local compute limits and gain access to more powerful, scalable LLMs

Thank You!